

Neural Symbolic Approaches for Legal Reasoning

Michaela Blomeier

March 29, 2026

1 Introduction

Legal Reasoning by Artificial Intelligence is not only theoretically of interest when wanting to test limits and capabilities of AI systems, but has also many potential applications in the field of law practice, law education and explainability for non-lawyers. An AI system with adequate legal reasoning ability could aid lawyers in document review and summarization, thus reducing time costs and help structure argumentation processes (Surden 2018). This would also benefit law students in learning correct reasoning methodology and applications for non-lawyers are diverse as well, since automated methods for compliance checking and general explainability of legal statutes in easier to understand language would help reduce the complexity of law for firms and private individuals not specialized in the legal field.

To deploy AI systems capable of good quality legal reasoning there arise certain requirements:

1. **Understanding of natural language and legal language**
2. **Information extraction of relevant text documents:** For legal reasoning the AI system must also be able to extract relevant sections and information from all sorts of text documents, e.g. supporting evidence which could be non-law related or specialized in a different field entirely
3. **Accounting for interpretive openness of legal sources and in human and judicial reasoning:** Legal statutes often have purposely vague formulations which require interpretation, often by court rulings regarding related cases which have to be considered by the AI system as well.
4. **Easily update the legal knowledge base:** Since laws change all the time and very rapidly, a legal AI system must be able to update its legal knowledge and ideally this should be computationally inexpensive.
5. **Methodical correctness:** Under methodical correctness we understand the principle that the AI system has adequate logical understanding which is the basis for legal reasoning and therefore correctness of reasoning methodology. Basic logic understanding comprises concepts such as if a certain fact A holds then it also has to hold that the fact not A does not hold since these are logically equivalent. This is especially critical, because we do not want the AI system to contradict given knowledge bases, such as law statutes.

The Paper of (Calanzone, Teso, and Vergari 2024) shows examples that Large Language Models often struggle with answering Yes-No questions if the question is posed as a negation and therefore shows a missing concept of logical understanding in LLMs. The Llama-3 8B model was queried with the questions "Is a computer an airplane?" and "Is it true a computer is not an airplane?". In the first case the LLM answered correctly with "no", in the negated formulated question the model responded falsely with "no".

6. **Consistency of analysis and outcome:** If one has given identical facts and one applies the same correct legal reasoning to these facts, then one should always arrive at the same conclusion.

The Paper of (Kant et al. 2025) asked language models to evaluate if claims were covered by a given insurance policy and each question was given to the model 10 times for evaluation and in both zero-shot prompting and chain-of-code prompting the language model answered only some of the 10 times correctly and the rest of the times falsely, hereby demonstrating that LLMs do not answer the same question with given identical facts identically, even with chain-of-code prompting.

7. **Transparency of reasoning process:** In law the reasoning process with which you arrive at your conclusion is often just as important as the result itself and the reasoning process needs to form a justifiable logic chain to be valid and also auditable and retraceable by anybody trying to comprehend.

Studies mentioned in (Tan et al. 2024) observe that LLMs deliver correct answers even when having errors in their reasoning process. This shows a misalignment of the reasoning ability of the LLM and the reasoning outcome.

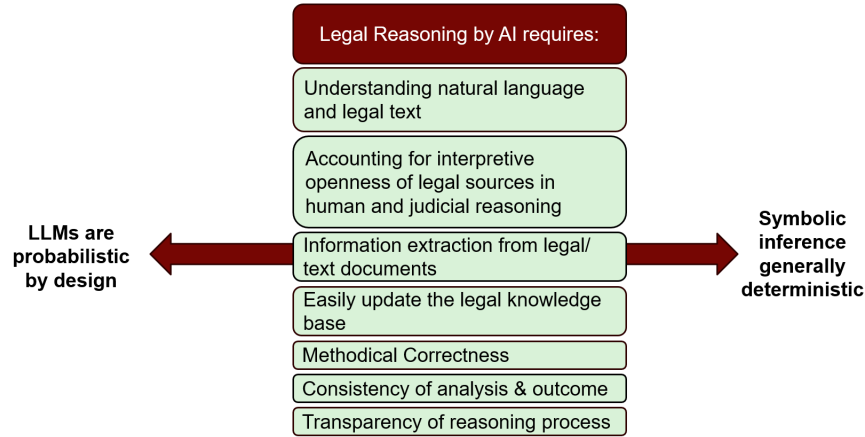


Figure 1: Our defined criteria for legal reasoning by AI. Use Neural-Symbolic AI to combine the strengths of both approaches.

1.1 Neural AI based systems for law applications

Since LLMs are probabilistic by design, autoregressively generating the next token, they have become very good at natural language understanding, information extraction and summarization. What they largely struggle with are the 3 points of methodical correctness, consistency of analysis and outcome and transparency of reasoning process. Depending on the system architecture an easy to update knowledge base poses a significant challenge as well.

There exist many neural approaches to enhance interpretability of legal reasoning of Large Language Models, such as special prompting techniques like Chain of Logic (Servantez et al. 2024). Similar to Chain of Thought Prompting, Chain of Logic gives examples of the decomposition of the reasoning process in the prompt to answer the question, but in the format of the IRAC framework more suited to legal reasoning: Spotting the issue, identification of rule, application of rule and conclusion. Specifically, rule application can be decomposed into smaller elements for independent solving and then recomposed back together to arrive at the final conclusion. Other methods for enhancing legal knowledge of LLMs include of course supervised fine tuning for the legal domain and legal-oriented pre-training such as done in combination in (Zhou et al. 2024) or just further pre-training on legal data or pre-training from scratch on legal data in (Chalkidis et al. 2020).

1.2 Symbolic AI for law applications

Very popular before the 2000s was Symbolic AI, mainly based in logic rules and derivations thereof, making these methods generally deterministic and good for reliability and transparency of reasoning. (Valente, Breuker, and Brouwer 1999) presents their approach to modeling legal knowledge with the computational framework ON-LINE (Ontology-based Legal Information Environment) for reasoning with legal knowledge and retrieving legal information. It is based on the idea of separating the legal text source in natural language text and legal knowledge base which is represented with a specific constructed ontology. Both sources are connected with modal links to allow for traceback of definitions to their sources and actual reasoning is only performed computationally on the ontology. Modern approaches rely on symbolic representations as well, such as (Cummins et al. 2025), which transforms insurance contracts into codified representations in the special language InsurLE, a subset of Prolog, to help with many applications, for example automated claims handling and efficient contract management. However, Symbolic AI struggles with natural language due to ambiguity.

1.3 Neural-Symbolic AI

A natural conclusion to draw from this, is that since both of the strengths of symbolic and neural approaches complement each other, we could explore an AI system combining both approaches for legal reasoning, so a neural-symbolic AI system, to be able to try to fulfill our 7 stated criteria in figure 1.

In the following seminar paper we will be providing an overview of current neural-symbolic approaches to legal reasoning, but also neural-symbolic approaches which were originally not applied in the legal field, but whose methodology could be transferred to legal reasoning as well.

Neural-Symbolic AI contains two parts: The **neurological component** and the **symbolic component**. The neurological component is a neural network of any architecture and in our application mostly LLMs. The symbolic component we define to be reasoning procedures over explicit formal structures with defined rules, e.g. over graphs or using rules of a certain logic. We specifically do not restrict the definition only to logic and allow any formal structure here, since there are quite a large section of approaches incorporating graph structures, such as knowledge graphs. Neural-Symbolic AI is therefore an AI system combining both a neurological and symbolic component in some form which jointly contribute to the output.

This seminar paper gives an overview of current approaches in the field of neural-symbolic AI through abstractly grouping them into categories in section 2 and then showcasing each category in more detail with an example in section 3 whose results are then discussed in section 4. Section 5 focuses on potential future work directions in the area of neural-symbolic AI for legal reasoning and lastly section 6 draws some conclusions on performance and applicability to legal reasoning.

2 Overview of Current Approaches

We decompose approaches into the interactions of neural and symbolic components and group them into categories based on methodologies, as depicted in figure 2. Each individual method has its own approach to how reasoning is carried out and correctness is evaluated and this overview simply categorizes them abstractly. In the following we will elaborate further on the individual categories:

1: Mapping on to a Formal Structure

This class of approaches takes all relevant natural language information, legal statutes, court cases and various other text documents and maps them on to a different representative structure, which could be code of a certain programming language, a type of graph, some ontological structure based on a logic or any kind of similar representation. Inference and reasoning is therefore done on this formal structure to produce the result. The main benefit to gain from this methodology is that reasoning on the representative structure is more deterministic and step-by-step verifiable than purely prompting a LLM, since reasoning steps and

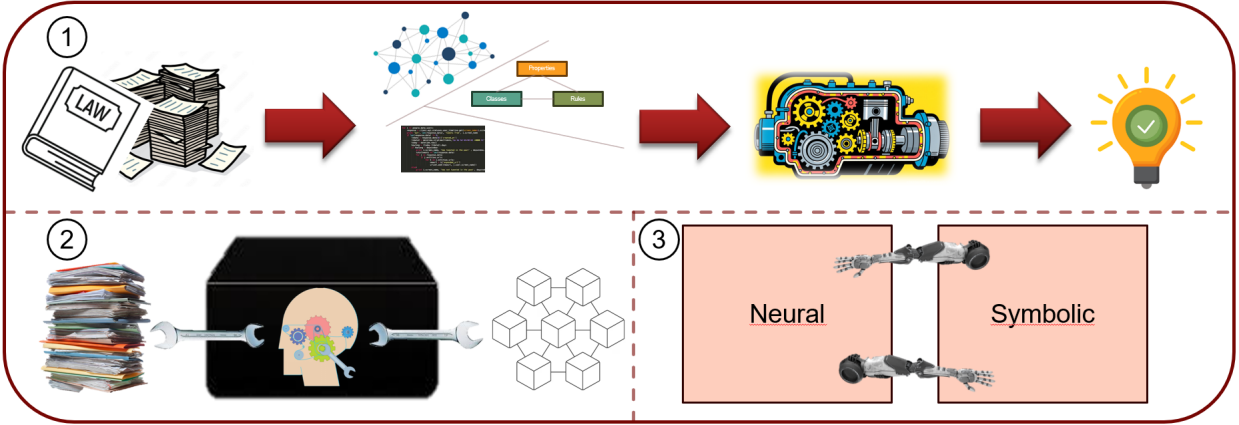


Figure 2: High level abstraction of neural symbolic pipelines in legal reasoning

decisions are made visible. The 3 subcategories depicted in the overview graphic will also be the ones we focus on in this seminar paper since they are the ones found in the literature, however there might also exist methods with differing structures.

2: Fine Tuning

There exist various approaches to fine tuning a model to improve its abilities with regard to the task. Here fine tuning is done with data from both language resources and some sort of incorporation of symbolic data, such as code examples simulating reasoning steps or incorporating logic statement correctness into the loss function.

3: Neural and Symbolic Interaction

This is among the three the smallest category. It represents methods which approach the whole task as a collection of many sub tasks which can be solved individually and for each sub task it is chosen if a neural or symbolic approach solves the task most appropriately and the individual results form the inputs to the next parts of the system. In general neural-symbolic AI there also exist methods where for example neural networks help constrain the search field of the symbolic component, so they work in each others operating domain.

Figure 3 organizes the literature of this seminar paper into the categories discussed above.

3 Detailed Analysis

In this section we will showcase specific approaches as found in the literature. Firstly we give a general overview of the section of law each approach focuses on and which models and datasets were used for evaluation. Secondly, following our definition of Neural-Symbolic AI, we show how in each architecture the neural and symbolic component are individually built and then come together to output results. Lastly we assess the methodologies for legal reasoning by AI by interpreting numerical results and assessing the architecture.

3.1 Category 1: Mapping on to a Formal Structure

As explained in the overview section, we divide the first category into 3 subcategories based on the different formal structures used. After the analysis of each individually we come back to compare each of them all together.

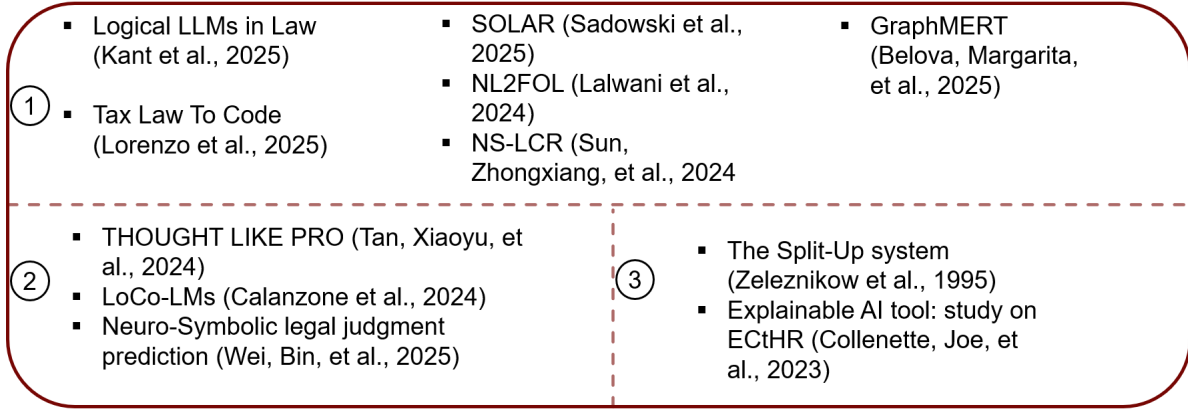


Figure 3: Literature papers organized into their respective categories

3.1.1 Ontological Structure

We focus here on the paper of (Sadowski and Chudziak 2025): “On Verifiable Legal Reasoning: A Multi-Agent Framework with Formalized Knowledge Representations”.

The application is focused on tax liability calculations based on US federal tax statutes, so given certain situations and American tax law sections, their Neural-Symbolic AI system consisting of multiple agents needs to calculate the correct tax amount to be payed. The evaluation of the model performance is done on the SARA (Statutory Reasoning Assessment) dataset (Holzenberger, Blair-Stanek, and Van Durme 2020) of LegalBench (Guha et al. 2023), a whole collection of datasets used to assess many different reasoning abilities of AI systems regarding law. The SARA dataset is rather small, there are only about 100 entries each consisting of taxpayer information and relevant tax code sections needed for the model to arrive at the answer. The general approach is similar to how a student would learn law since the whole procedure is divided into two steps: The Knowledge Aquisition Phase and the Knowledge Application Phase, together forming their multi-agent framework SOLAR (Structured Ontological Legal Analysis Reasoner). It focuses on acquiring general law knowledge of US tax law first and only then applies it to specific case facts, e.g. those given in the SARA dataset.

In figure 4 one can see on the left the framework for Knowledge Aquisition from statute text and on the right the Knowledge Application phase to a user query.

The core symbolic component of this multi-agent framework is the ontological structure, a tuple of 3 sets consisting of classes, properties and rules based on First-Order Logic. This tuple is named the T-Box, with classes representing legal entities, e.g. Taxpayer, or concepts such as Income. Properties encode relations between classes, these can be unary, e.g. isMarried(Taxpayer), relationships, e.g. isMarried(Taxpayer1, Taxpayer2) and datatype attributes, e.g. IncomeAmount(Taxpayer1, amount). Finally we have the set of logic rules in First-Order Logic. This T-Box encodes the legal knowledge of the model in general form, whereas in the Knowledge Application Phase this T-Box is populated with information of real case facts, so general variables, such as Taxpayer1 would become a real person such as Alice. If the T-Box is filled with case facts of the user query, (Sadowski and Chudziak 2025) refers to this as the A-Box. If one has all these rules given in First-Order logic, new statements can be derived from these implications simply by applying logic inference methods, here an SMT solver was used to add new inferred statements to the general Knowledge Base of the model. To perform the actual tax calculations they constructed the T-Box Interpreter, a Python function which calculates tax amounts and percentages if the legal requirements are given, which are stated in the T-Box or newly derived from the previous knowledge base.

The task of the neurological component, the LLMs, is to translate statute text to the form of the T-Box in the Knowledge Aquisition Phase and in the Knowledge Application Phase to map the user query on to the structure of the T-Box to create the A-Box. The LLMs extract the classes, properties and rules and also

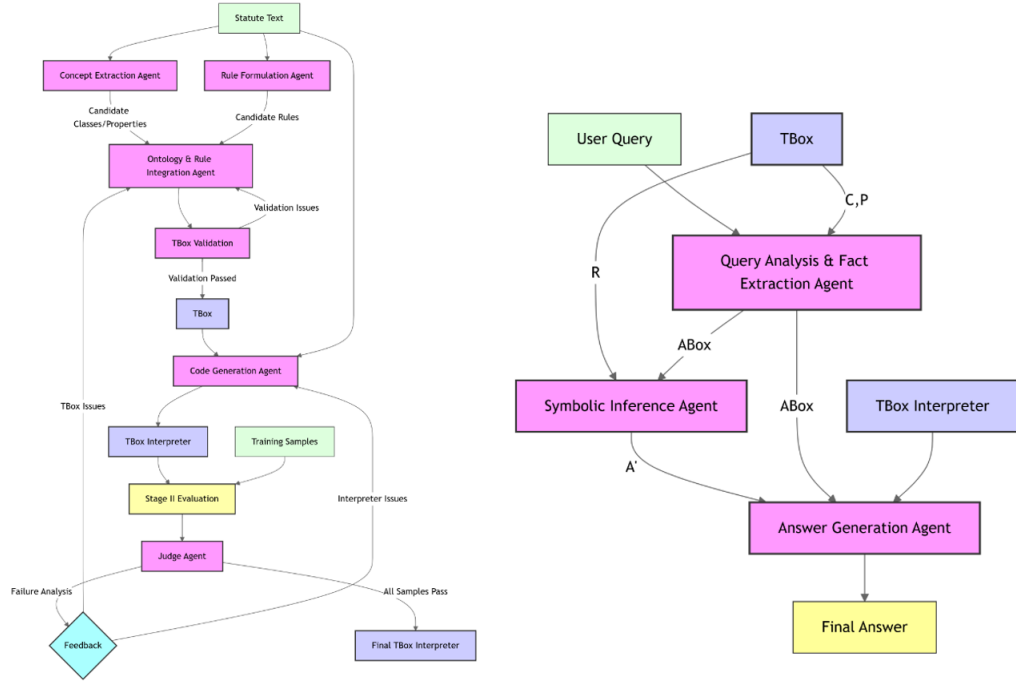


Figure 4: Graphic from (Sadowski and Chudziak 2025). On the left is the graphical overview of the multi agent system of the Knowledge Aquisition Phase and on the right the Knowledge Application Phase.

generate the code making up the T-Box interpreter from the T-Box. There are several steps where LLMs are applied here to mold the natural language in to the structure of the ontology to formally reason over, hence the name multi-agent framework since several agents appear at different steps of the pipeline. They also deploy verification agents to check that there are no logical contradictions in the T-Box and that terminology is consistent. The final output of tax amount is therefore a joint result of the A-Box and formally derived new facts which activate the calculation in the T-Box interpreter.

For evaluation (Sadowski and Chudziak 2025) applies several LLMs in the Knowledge Application Phase along with one previously generated T-Box by the o4mini-model. This purposeful choice allows the evaluation of results, measured as accuracy on the SARA benchmark dataset independent of the quality of the generated T-Box. Several models are used for evaluation, for example GPT-4.1 mini and DeepSeek V3. The modeling of natural language therefore comes from the basis of LLMs used and legal concepts and legal language is modeled by the mapping of language to ontological structure, done by extracting classes, properties and rules in First-Order Logic by the LLMs. (Sadowski and Chudziak 2025) mention specific errors occurring in the pipeline, demonstrating trouble in the modeling of legal language and concepts. The most fundamental error was the TBox containing insufficient vocabulary to represent all needed legal concepts to properly calculate tax amount owed, therefore leading to a wrong result even with correct reasoning process which would be the fault of the concept extraction agents. Another category of failure was the incorrect implementation of the calculation logic in the TBox interpreter even though the underlying ontology represented the legal precedence correctly, leading here to errors in tax amount as well, due to problems of the code generation agent. The paper applied all LLMs with 3 different techniques on the evaluation dataset: zero-shot prompting, chain of code prompting and their own framework SOLAR. It is shown that a model with weak zero-shot accuracy (15.63%) and chain-of-code accuracy (22.50%) such as DeepSeek V3 has an accuracy with SOLAR of (82.30%), whereas an already strong reasoning model like o4 mini has zero-shot accuracy (91.7%) and chain-of-code accuracy (93.80%) and a SOLAR accuracy of (87.50%). Generally over the

numerical results it is seen that the SOLAR framework improves performance of weaker models in zero-shot prompting substantially, but LLMs with already pretrained strong reasoning ability have a slight decrease in accuracy with the SOLAR framework. This points to a possibility that the mapping on to the ontological structure for reasoning in First-Order Logic restricts the original model’s reasoning ability more severely, since law itself is more expressive than First-Order Logic, leading to a reduction in performance accuracy. The main advantage of the proposed multi-agent framework, as mentioned in (Sadowski and Chudziak 2025), still lies in the transparency of reasoning pathways through the step by step processing pipeline and allowing for inspection when needed.

3.1.2 Code of a suitable Programming Language

The focus of this subsection will be on the paper of (Lorenzo, Pietromatera, and Holzenberger 2025) “Translating Tax Law to Code with LLMs: A Benchmark and Evaluation Framework”. The goal is to reliably translate legal language, focused on french tax law, into computer executable code written in the Catala programming language specifically designed for tax law. To assess the quality of translation from legal text to code, they propose a set of metrics to get indicators from many different perspectives. For training (Lorenzo, Pietromatera, and Holzenberger 2025) use the publicly available dataset for the Catala programming language consisting of legal provisions needed to be translated into code along with helpful metadata, e.g. legal concepts, which are already written in code to aid the model and a reference output of the code translation generated by computer scientists and lawyers.

The symbolic component consists of the explicit program in Catala which can be executed and provides transparency and also the ability to inspect the execution at any point of the whole reasoning process. Just as seen in the previous section, the neural component, so the chosen LLMs are responsible for mapping legal language to Catala Code.

A central result of the paper is that LLMs are indeed capable of generating syntactically valid and semantically meaningful Catala Code and their results show a qualitative jump in comparison to other previous approaches, which most likely stems from the choice of using the Catala language specifically designed to align with legal language (Lorenzo, Pietromatera, and Holzenberger 2025). As the authors (Lorenzo, Pietromatera, and Holzenberger 2025) point out though, the main obstacle to appropriate modeling of legal concepts and legal language is the mapping of legal text to abstract code. In particular, there arise interpretation-related issues which cannot be fully represented in code, since programming requires to committing to a single, precise definition or interpretation contrasting necessity for multiple interpretation possibilities in legal language as stated in the introduction section. Another challenge the authors point out consists in the construction of these training datasets for translating the language of legal reasoning into code, a very high-level task requiring human experts which is extremely time-consuming and labor-intensive. Additionally another limitation in human collaboration on the creation of such datasets is that inconsistencies can arise, since different experts may translate the same legal provision slightly differently requiring more effort on agreement of correct representation.

3.1.3 Graph Structure

The paper of (Belova et al. 2025): “GraphMERT: Efficient and Scalable Distillation of Reliable Knowledge Graphs from Unstructured Data”, proposes an approach to distill text corpora into high-quality Knowledge Graphs using their GraphMERT pipeline with an application in the biomedical field, specifically diabetes. The idea of (Belova et al. 2025) encompasses two different sources of information. There is the syntactic domain knowledge from a large text corpus and semantic knowledge about the application area encoded in a knowledge graph. The seed knowledge graph used in (Belova et al. 2025) was taken from the UMLS Metathesaurus and for syntactic text, relevant papers to the application topic were chosen from published MEDLINE journals. To create a rich knowledge graph out of the seed knowledge graph, (Belova et al. 2025) propose the structure of a leafy-chain graph which gets populated by searching for matching entities in the syntactic text and UMLS graph and linking entities with corresponding relations using the helper LLM Qwen3-32B.

Figure 5 gives an overview of the knowledge graph creation framework.

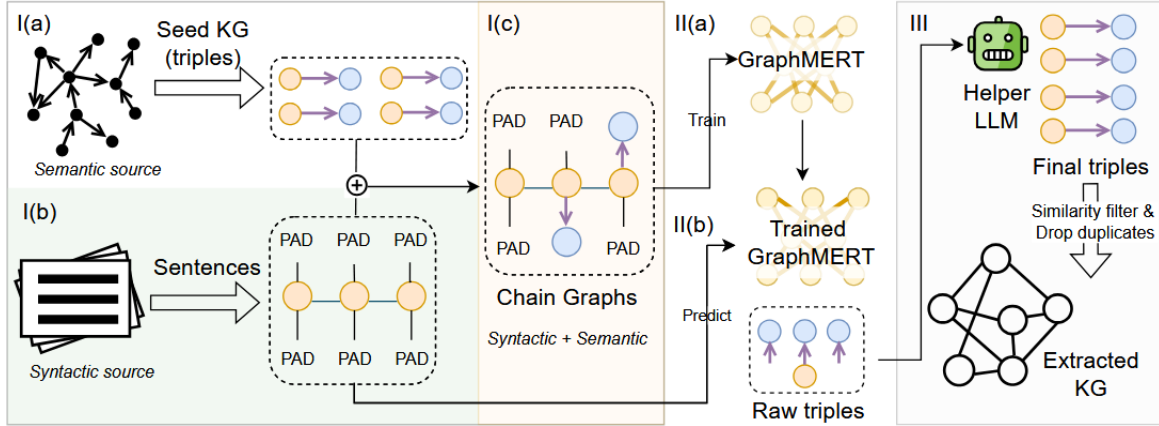


Figure 5: GraphMert knowledge graph creation pipeline from (Belova et al. 2025)

Step I is the merge of syntactic source with the semantic seed knowledge graph. In the knowledge graph each triple consists of the head, relation and tail and both the syntactic and semantic source are fused into the initial leafy chain graph structure by matching triples based on embedding similarity and char-3-gram matching. In step II the whole graph is trained similarly to a transformer architecture with Masked Language Modeling and Masked Node Modeling where the next word is masked and the architecture has to predict the next word given the whole context, but also linked heads are masked and need to be predicted given the whole graph. Finally in step III a rich knowledge graph can be extracted by querying the model to predict the head of an entity in the syntactic text corpus with a given relation and select among the top k options the best fitting into the context using a helper LLM. Evaluation of the framework is done by measuring individually the whole quality of the constructed knowledge graph by using it as the only information source to run GraphRAG and evaluate the resulting performance on question answering on specific datasets. The paper uses two different scores to rate separately the quality of the generated triples independently of the performance of the whole graph.

For this analysis we look at the numerical results of the whole knowledge graph evaluation at question answering on benchmark datasets, done on the MedMCQA, MedQA and medical MMLU dataset. It can be seen that the GraphMERT framework regularly outperforms the baseline LLM generated knowledge graph measured in accuracy: The baseline LLM KG achieves 72.1%, 85.3% and 71.0% on the MedMCQA, MedQA and medical MMLU respectively, whereas GraphMERT achieves 73.8%, 88.0% and 74.7% It also outperforms the seed knowledge graph on the medical MMLU dataset and MedQA dataset, which only achieves 73.1% and 81.3% respectively, but has a lack in performance compared to the seed knowledge graph on the MedMCQA dataset with the seed knowledge graph achieving 76.5%. So overall accuracy improvement over baseline and seed knowledge graph is only slightly by 1% – 6%. While the improvements over the baseline are modest, the whole GraphMERT framework has only about 80M parameters which make it quite computationally efficient and could highlight the effectiveness of incorporating symbolic structure into the reasoning with limited capacity.

In principle this approach could be translated to law as well since the text corpus used as syntactic source can easily be exchanged for any law text, but one would need a seed knowledge graph as semantic source with rich meaningful relations in the field of law most likely crafted by domain experts which is extremely costly and time consuming. In addition the GraphMERT framework relies on predefined relations since the helper LLM only links entities and does not propose or introduce new relations. In such a model one would also have to verify the quality and validity of such newly generated relations which would require human verification again. Legal concepts would also be tricky to handle here since there would have to be a way

to incorporate the handling of open-textured meanings and encode the concept that some legal rules take precedence over others or some statutes override others in cases of conflict.

3.1.4 Comparison

The three approaches discussed in this subsection illustrate different strategies for integrating neural language models with symbolic representations in legal reasoning tasks. While all approaches rely on LLMs to interpret natural language, they differ substantially in the form of symbolic structure used for reasoning. The SOLAR framework (Sadowski and Chudziak 2025) maps legal language onto an ontological structure expressed in First-Order Logic, enabling formal reasoning through a T-Box interpreter. In contrast, the approach of (Lorenzo, Pietromatera, and Holzenberger 2025) translates legal provisions directly into code written in the domain-specific language Catala, thereby embedding legal reasoning into an executable computer program. Finally, the GraphMERT framework (Belova et al. 2025) focuses on distilling structured knowledge graphs from textual corpora, where reasoning emerges from graph-based representations combined with neural embedding models.

Despite these methodological differences, all approaches face a common challenge: the reliable mapping of open-textured legal language into a formal representation suitable for automated reasoning. Errors may arise at different stages of the pipeline, such as incomplete ontological vocabularies in SOLAR, ambiguities in translating legal provisions into executable code, or dependence on predefined relations when constructing knowledge graphs. On the other hand each approach offers increased transparency compared to purely neural models, as the reasoning process can be inspected through ontological rules, executable programs, or graph structures. The table in figure 6 gives a compact comparison.

<u>Approach</u>	<u>Symbolic Representation</u>	<u>Role of LLM</u>	<u>Main Advantage</u>	<u>Limitation</u>
SOLAR (Sadowski and Chudziak 2025)	Ontology with First-Order Logic in form of T-Box and A-Box	Extract classes, properties and rules and generate T-Box interpreter	Transparent reasoning	Errors in ontology construction
Catala Code (Lorenzo, Pietromatera, and Holzenberger 2025)	Catala Code	Translate legal text into executable computer program	Deterministic execution of a computer program	Difficult to encode varying legal interpretations
GraphMERT (Belova et al. 2025)	Knowledge Graph	Extract entities and link to relations	Knowledge representation is inspectable and easily auditable	Requires predefined relations and quality seed knowledge graph

Figure 6: Comparison table of approaches in section 3.1

3.2 Category 2: Fine Tuning

In our next category of fine tuning we introduce: (Tan et al. 2024): “THOUGHT LIKE PRO: Enhancing Reasoning of Large Language Models through Self-Driven Prolog-based Chain-of Thought”. The paper does not explicitly focus on the field of law, but their methodology can easily be applied to legal data as well. To enhance reasoning abilities of LLMs by teaching them step by step reasoning, (Tan et al. 2024) perform supervised fine-tuning of the Llama3-8B-Instruct model on the following datasets which they call the THOUGHT LIKE PRO framework. We list all datasets in detail to clearly describe which datasets were used for training and which for evaluation.

These 3 first datasets are used to train and also evaluate model performance:

- GSM8K: contains 8,500 grade school math word problems made by human experts and for the training of the THOUGHT-LIKE-PRO framework they sampled 2,000 entries for training.

- ProofWriter: has 482 small knowledge bases in English containing rules and facts and corresponding questions regarding the knowledge base which can be definitely proven as true or false. For training 100 knowledge bases and their questions were sampled for training and the rest used for evaluation.
- PrOntoQA: a dataset containing synthetic entries of question and answer pairs with a context of a short story and to answer the question the model needs to chain several reasoning steps. 100 samples were randomly sampled from the “Fictional Characters” section and the remaining 400 used for evaluation.

The rest of the datasets are only used for evaluation, so the model has not seen any of these during training and are used to test Out Of Distribution (OOD) behaviour to see if the model has learned general reasoning abilities through the training regime:

- MMLU: knowledge based question-answering encompassing 57 subjects, such as STEM, humanities, social sciences. The entire dataset was used for evaluation.
- GPAQ: 448 multiple-choice questions in disciplines such as biology, physics and chemistry. Again the entire dataset was used for evaluation.
- HumanEval: The entire dataset, comprised of 164 programming problems, used for evaluation.
- MATH: Whole dataset, consisting of 12,500 competition-level mathematics problems, used for evaluation.

In the numerical results on the OOD datasets it can be seen that the THOUGHT LIKE PRO framework does not outperform the baseline model performance except in the MATH dataset. The cause is possible to lie in the fact that the GSM8K dataset already consists of high level math problems which was used during training, so the model would already have been familiar with these sort of questions.

The framework mainly consists of 2 steps which are the construction of the fine tuning dataset from the training dataset with help of the Prolog reasoning engine (Steps 1 to 3 of figure 7) and supervised fine tuning (Step 4 of figure 7), so maximizing the log-likelihood.

Figure 7 shows the entire pipeline of the THOUGHT LIKE PRO framework.

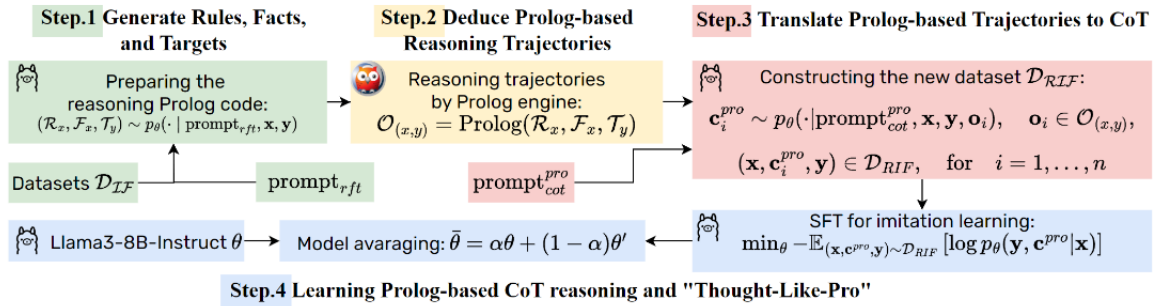


Figure 7: Fine Tuning framework from (Tan et al. 2024)

The symbolic aspect of this fine tuning regime lies in the use of the Prolog programming language, which does rule-based reasoning with Horn clauses. If one has basic facts, rules and given targets one can use a Prolog reasoning engine to find a reasoning path to get from facts to our target utilizing the rules, thus symbolically proving the result using Horn clauses. These generated reasoning paths are utilized to help fine tune the base model.

Given one of the training datasets with question and answer pairs from which (Tan et al. 2024) can extract rules, facts and targets representing this context in Prolog format by using a specially constructed few-shot prompt of (Tan et al. 2024) to the LLM. After this, as explained in the symbolic paragraph above, one can

utilize a Prolog reasoning engine to derive multiple reasoning paths from facts to targets. As mentioned in (Tan et al. 2024) one cannot directly derive several reasoning trajectories at the same time, one is required to employ a meta solver to find several. Since reasoning trajectories are given formally in Prolog, a natural language Chain-of-Thought can again be extracted by applying a specifically designed few-shot prompt of (Tan et al. 2024) to the LLM and therefore forming the fine-tuning dataset D_{RIF} with each entry consisting of the question x , the answer y and the Chain-of-Thought in natural language c_i with $i = 1, \dots, n$, one each for the different possible trajectories. The fine-tuning step consists of minimizing the negative log-likelihood and employing the model averaging technique to counter catastrophic forgetting when fine-tuning.

The paper follows two main questions: Does this fine-tuning regime improve model performance on reasoning tasks over baseline model performance and if the model is represented with multiple reasoning trajectories during training, does this show improvement over only a single reasoning trajectory presented, similarly to the approach that people learn and understand a subject in more depth if they have seen several different methods to arrive at the solution.

On datasets the model has seen before, the proposed framework achieves higher accuracy than the baseline, particularly when multiple reasoning trajectories are used. This supports the hypothesis that including multiple symbolic reasoning paths improves performance on tasks aligned with the training distribution.

In contrast, on the out-of-distribution benchmarks, the baseline model outperforms the fine-tuned variants with and without model averaging as well as with multiple and single reasoning trajectories, though only by a very small margin. This indicates limited transferability of learned reasoning during training to tasks beyond, but is also an argument that model averaging does counter the catastrophic forgetting effect and also smooths sharp task-specific adaptation of the fine tuning since performance of baseline and fine tuning with model averaging performances achieve nearly the same accuracy and have a difference of less than one percent.

Although THOUGHT LIKE PRO introduces an explicit symbolic part through Prolog programs, when querying the model the overall reasoning process still remains probabilistic and possibly sensitive to the prompt, so although methodological correctness might have improved through fine-tuning, transparency and reproducibility still raise concerns which also limits transferability to legal reasoning. Another issue when applying this framework to legal reasoning could be if Horn-clauses used in Prolog can sufficiently capture the modeling of legal reasoning which are used to produce the reasoning trajectories in the dataset for fine-tuning. In fine-tuning regimes updatability is also a common problem, since laws change often and therefore data used for training could become outdated requiring one to completely retrain the model once again with updated data which could be quite time intensive.

3.2.1 Related Approach

The system of (Wei et al. 2025) integrates symbolic reasoning with neural learning in an elegant way, by optimizing a loss function including a term measuring the discrepancy of neural output and the judgment of the logic module which achieves its result by applying First-Order logic rules. The proposed approach focuses on the task of legal judgement prediction for civil cases in context of Chinese private lending law.

To extract fact elements from the fact description and claim (Wei et al. 2025) applies a GPT-4 model and 5 rules in First-Order Logic are formulated manually by hand, extracted from legal articles that have been observed to hold. These rules consist of an antecedent of logical conjunctions and a precedent of the claim being supported or partially supported.

The neural component receives the same fact description and claims as input and predicts judgement outcomes based on a neural network consisting of a pre-trained word-embedding model, two Bi-LSTM layers and a co-attention layer between facts and claims for claim-aware representation of facts and fact-aware representation of claims which improves judgement performance (Wei et al. 2025). Final judgement predictions are obtained after a softmax-layer. The total loss is a sum of the neural components cross entropy loss and the discrepancy loss which measure how similar the output of the neural component and the symbolic logic module is which encourages the neural network to produce predictions consistent with the logical reasoning of the logic module.

Experimental results of (Wei et al. 2025) show that the proposed approach slightly outperforms existing state-of-the-art methods. The GPT-4 based fact extraction component demonstrates strong performance in identifying relevant factual elements, although limitations remain in handling nuanced legal and calculation details, suggesting use of external calculation tools to be helpful. Similarly (Sadowski and Chudziak 2025) employed the T-Box interpreter, a Python function, instead to calculate tax amounts separate from the neural component. A further limitation in the design are the logical rules extracted manually allowing for descriptive precision to the domain, but limiting scalability of framework without additional expert effort.

3.3 Category 3: Neural and Symbolic Interaction

The paper of (Zelevnikow and Stranieri 1995) develops the so called Split-Up system which is a framework to predict the outcome of property distribution among spouses following divorce under Australian family law. To determine what each spouse gets, all assets are jointly considered, so thrown into one pot and according to the percentage, it will be determined which party receives which share, also called the percentage split determination. To train the model, (Zelevnikow and Stranieri 1995) put together a training set of 150 unreported cases of the Family Court of Australia for their model.

The idea behind this approach uses the following two concepts:

1. **Toulmin Argument Structures:** Toulmin Argument Structures follow the idea that all arguments have a structure presentable with these 4 following principles: claim, data, warrant and backing. The claim is an assertion that can be found given the data, e.g. in our application case one could derive the claim that the marriage could be considered about average in wealth after observing income and value of assets. The justification of the claim aims to say how the claim follows from the data. The backing legitimizes the warrant by referencing a specific case or statute, explaining why exactly the warrant is acceptable. So the warrant justifies the claim from the data and the backing backs that the warrant itself is correct by referencing the law. (Zelevnikow and Stranieri 1995) here only use these 4 structures, there exist further extensions with rebuttal and modality which are not applied here.
2. **Conclusion and Explanation can be done separately:** (Zelevnikow and Stranieri 1995) give a very illustrative example. To show why 196 divided by 14 is 14 one can either show the school division algorithm step by step to see how one arrives at the result. This is very close to the principle of starting at the beginning and reasoning step by step until one arrives at the conclusion. The other way to prove this, is to show that 14 times 14 results in 196, so coming from the other side and working back from the claim to justify the result. This backwards working approach was also implemented in the model of (Zelevnikow and Stranieri 1995), since simple feedforward neural networks, as were used in 1995, but also modern deep neural networks do not explicitly say why the result they arrive at is such, they just produce the result and we can use this result of the neural network to be the claim and backtrack to produce an explanation why this claim holds.

The whole task is to determine the percentage each spouse receives, so to determine a percentage split such as 65% – 35% or 40% – 60%. The approach is to model the percentage of one party, e.g. the husband, with a neural network. The percentage that the other party then receives is then also determined. Certain contributing arguments will be the input to the neural network, such as the wealth argument or contribution argument as seen in figure 8, and the output will be a percentage category the husband receives. A nice illustration of this neural network proposal can be seen in figure 9. Important to note here is also that this feedforward neural network is not very complex in architecture, it consists of 3 layers with 16, 60, 13 neurons each. There are around 64 smaller arguments determined by experts contributing to the percentage split argument, so the whole problem can be split up into smaller subproblems of showing the validity of these individual small arguments. Then these individual smaller problems can again be determined either rule based or again with a small neural network, e.g. deciding if the marriage can be considered in one of the categories of very wealthy, wealthy, about average, less than average or far less than average in wealth.

There is one forward pass where from the data these 64 smaller arguments are determined either with a symbolic or neural approach, depending on which is suited better for the data and all these factors serve as

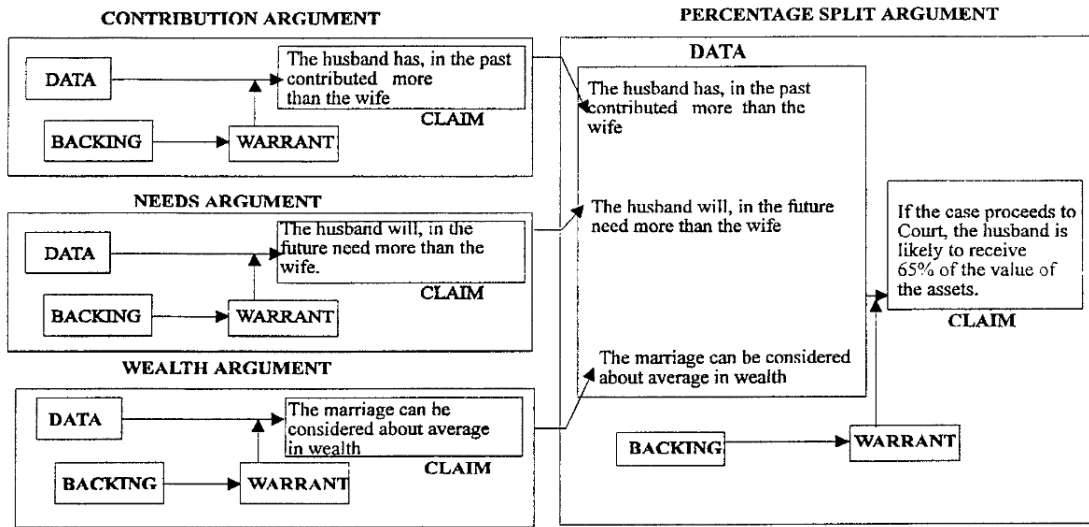


Figure 8: Toulmin Argument Structures in (Zeleznikow and Stranieri 1995)

input to the final percentage split neural network which then outputs the final percentage. The backward pass is to provide an explanation as to how the claim was inferred. Firstly one can determine the input arguments that counted as input to the neural network, e.g. the marriage was of average wealth or the husband in the past has contributed more than the wife, such as one can see in the highlighted points in figure 9. After retrieving the contributing arguments the system can further backtrack to show how these facts came about, by for example showing the expert rules used in the symbolic parts to determine the claim. If even further explanation is needed the warrant can be produced and if needed with the necessary legal backing. It is important to note here that these warrants are not created or written by the model, they were all already pre-written by experts and the system only selects those fitting the claim.

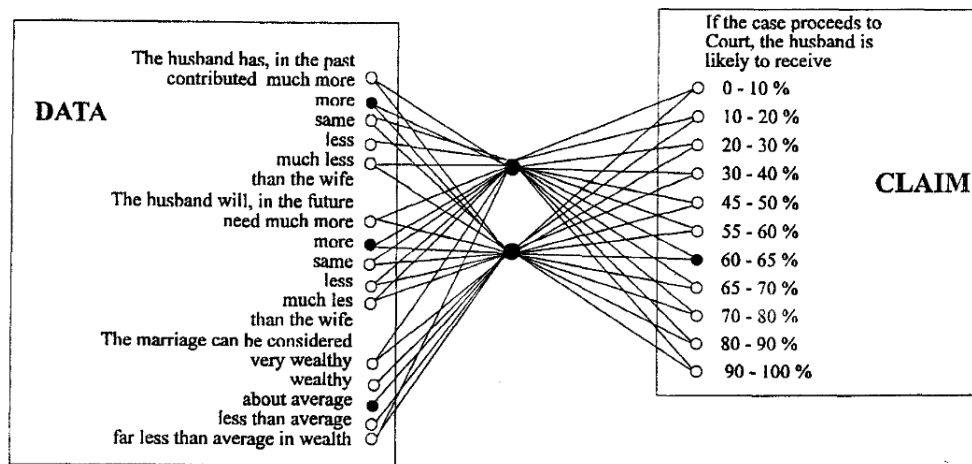


Figure 9: Final neural network in (Zeleznikow and Stranieri 1995) used to determine percentage split

One of the issues with this system that (Zeleznikow and Stranieri 1995) also point out is that the model can map to fine-grained categories and produce warrants why these categories should be justified, so it can determine a 60% – 65% split, but it cannot distinguish why exactly a spouse should receive 60% – 65% instead

of 65% – 70%. Also human legal reasoning does not derive deductively a precise percentage value of 64% instead of 61% either, the exact percentages emerge through adversarial argumentation of lawyers on behalf of their clients and judicial discretion. This also touches one of the points mentioned in (Zeleznikow and Stranieri 1995): An argumentation to why a certain claim should hold often is influenced by the HIWTCO (“How I want to come out”) principle. So the lawyer representing the wife and husband respectively base their argumentation from the perspective that they both want the highest percentage for their clients given the common asset pool and both will arrive at different arguments why their party should receive a higher percentage. This is why in legal reasoning by AI it is very difficult to just generate one unique explanation since in practice different lawyers can argue for different outcomes given the same factual basis.

This approach is very specifically engineered to the application in Australian family law. This is quite similar to the QUEEN approach (Quality assured efficient engineering of neural networks) ¹, which is an older method proposed in modern neural networks for increasing trustworthiness which describes the method to engineer special encodings tailored to the the data and only then use these to apply a neural network to. This is what the Split-Up System does, it has data and used experts to select the 64 contributing arguments to the percentage split and encodes representations which are fed into the final neural network. This suggests that Split-Up could benefit from re-application in modern settings with LLMs, just as how QUEEN-style approaches have been proposed for modern neural network applications.

A downside to this approach of engineering specifically to the problem setting and an explicit part of law is that it might be suited for its specific fixed task, such as determining the percentage, but not for general purpose legal questions, which people might have given certain situations and they would like to know what their legal rights are in this case.

3.3.1 Related Approach

A related approach of (Collenette, Atkinson, and Bench-Capon 2023) models legal reasoning with application to Article 6 of the European Convention on Human Rights which refers to the right to a fair trial. The framework aims to predict outcomes of pending applications and develops a tool supposed to aid in determining admissibility of cases. This is done using an ADF (Abstract Dialectical Framework) represented as a structured argument graph consisting of nodes and links to form a hierarchical reasoning structure resembling the structure of legal reasoning: Leaf nodes correspond to factual questions one can ask a user to obtain relevant case facts, so questions that can be answered with yes or no or a number between 0 and 1 to represent degrees of certainty. Further up in the graph the values of non-leaf nodes are determined by evaluating their children using specified functions and predefined acceptance conditions. Therefore each node can encode a specific logical function to determine whether the argument of the child node should be accepted in the parent node, giving an elegant way of handling conflicts between arguments of children nodes by encoding priority rules in the parent. The framework evaluates bottom to top in inference to give the outcome prediction in the root node of the graph propagated through from the bottom leaf nodes. After this pass a top to bottom evaluation then can give a structured representation of the argument on why the outcome prediction of the model holds and the whole reasoning process is implemented in Prolog for automated evaluation. An example of such a constructed ADF graph can be seen in figure 10.

The framework achieved high predictive accuracy with only one incorrect prediction among thirty evaluated cases. However, the approach relies heavily on a manually designed argument structure and therefore requires substantial expert effort to construct and maintain up to date which means its applicability to other legal domains may be limited. The structure could potentially be extended into a neuro-symbolic system by allowing acceptance conditions in nodes to be modeled by neural or symbolic components depending on the context. This decomposition of the task into smaller argument nodes is conceptually very similar to the Split-Up methodology where the legal decision is decomposed into smaller contributing factors which are individually evaluated and then aggregated into the final outcome and after the “forward pass” provide justification by doing a “backward pass” down the graph.

¹QUEEN approach discussed in lecture: Trustworthy Machine Learning Systems CIT4330017 at TUM

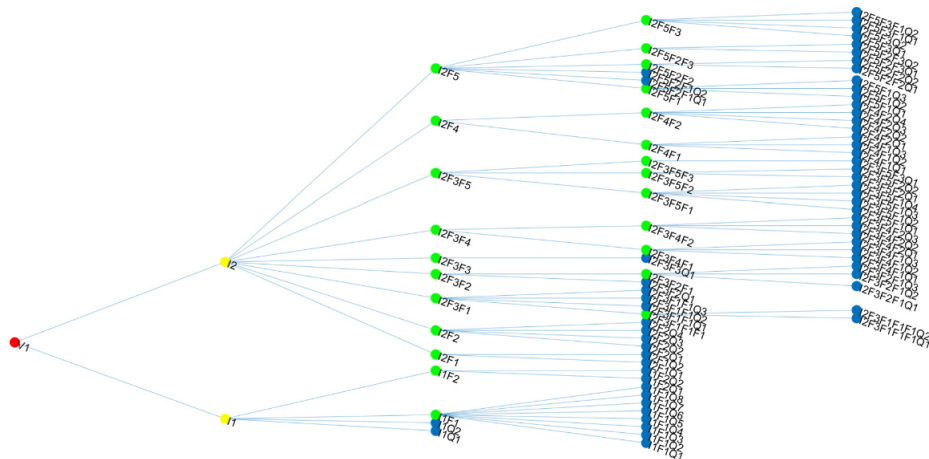


Figure 10: Example of final constructed ADF to determine admissibility of a case from (Collenette, Atkinson, and Bench-Capon 2023)

4 Discussion

This seminar paper surveyed neural-symbolic approaches to legal reasoning, by organizing them into 3 methodological categories:

1. mapping on to a formal structure
2. fine tuning strategies
3. neural-symbolic interaction

While these approaches aim to combine the strengths of neural and symbolic reasoning, their effectiveness must be evaluated with respect to the requirements of legal reasoning we gave in the introduction section and the following discussion analyzes these systems along the respective criteria.

4.1 Evaluation with Respect to Requirements for Legal Reasoning

Understanding legal language

Neural-symbolic methods improve the performance of smaller models with weaker reasoning capabilities, such as demonstrated in the SOLAR framework (Sadowski and Chudziak 2025). They introduce explicit symbolic structure through ontologies, substantially improving accuracy of models with very low zero-shot prompting performance on the small legal subdomain of tax law. This shows that the introduced formal symbolic reasoning compensates for deficiencies in smaller language models by enforcing specific legal rules that have to hold. This allows for weaker models to achieve performances close to large reasoning models in small subdomains which could pose potential if large computational resources are not available.

Similarly this can also be seen in (Lalwani et al. 2025) which develops a 3-step framework for general logical fallacy detection by mapping natural language statements to an expression in First-Order Logic and then applying a SMT solver to verify correctness of the statement or providing a possible counter example. The framework of (Lalwani et al. 2025), NL2FOL, achieves with the smaller LLM of Llama-7B a substantially higher accuracy than the baseline of simply querying the LLM (41% compared to 63%). Similarly to SOLAR as well, the two larger models of the GPT-4 series show a decrease in accuracy when using NL2FOL (91% compared to 70% and 96% compared to 78%). This supports the hypotheses that the mapping on to First-Order Logic for reasoning restricts stronger models reasoning ability more severely than smaller models which in contrast benefit from the introduced structure. Since the evaluation only considered a limited number of models further experiments would be necessary to confirm whether this trend generalizes across models

and it also must be taken into account that there occur additional sources of error during translation from natural language to logic which could also contribute to decrease in accuracy.

The potential of introducing symbolic structure to help achieve similar results with less computational resources is also supported by results from (Belova et al. 2025) since the GraphMERT framework was only trained with 80M parameters and achieved similar and slightly better results than the knowledge graph generated by the baseline LLM.

Similarly one can also observe performance improvement in logic-based legal case retrieval systems instead of relying solely on semantic similarity between query and candidate cases as done in (Sun et al. 2024). Retrieval is performed by matching similarity of legal logical structure of query and candidate cases therefore including structural semantic information, improving retrieval performance as seen in (Sun et al. 2024).

Methodical Correctness

As stated in the introduction methodological correctness encompasses basic logic understanding for legal reasoning, which is not always given as demonstrated in the following paragraphs. The fine-tuning based approach THOUGHT LIKE PRO does not consistently outperform the baseline of the original pretrained model on out of distribution data. The fine-tuned framework only improves on familiar data. This suggests that improvement in a model with the fine tuning regime is due to specialization and not general improvement in reasoning ability, which could pose a problem in legal reasoning since new circumstances could arise that differ from prior seen examples.

An alternative strategy to supervised fine tuning is including logical constraints into the loss function as is done with semantic loss in LoCo-Lms (Calanzone, Teso, and Vergari 2024). The semantic loss function explicitly penalizes, if logical constraints are violated and numerical results of (Calanzone, Teso, and Vergari 2024) show that their fine tuned framework outperform supervised fine tuning approaches and the baseline model. A big limitation to the applicability to legal reasoning is their approach only goes explicitly to assigning absolute true or false values to atomic statements which with certainty can be judged as either true or false given a knowledge base which in legal reasoning over statutes is not always the case.

Across the discussed approaches, the primary sources of failure consistently arise not from the symbolic reasoning mechanisms themselves, but from inaccuracies in the neural stages responsible for constructing the symbolic representations on which the reasoning operates.

In the SOLAR framework (Sadowski and Chudziak 2025), one fundamental failure mode occurs when the generated T-Box lacks sufficient vocabulary to represent all relevant legal concepts required for calculating the tax amount. In such cases, the reasoning process may remain logically correct, yet still produce incorrect results due to an incomplete conceptual representation introduced by the concept extraction agents. Another source of error in SOLAR arises from incorrect implementation of the calculation logic within the T-Box interpreter, where the ontology may correctly encode legal precedence relations but the generated executable code fails to faithfully implement the intended logic, leading again to erroneous tax outcomes due to errors of the code generation agent.

Similar failure modes appear in the NL2FOL pipeline (Lalwani et al. 2025), where errors most frequently arise during the additional context extraction stage done by the “Background Knowledge Retriever”. The additional logic context in the natural language text is extracted by querying a helper LLM to produce additional logic statements given implicitly in the text to help with reasoning later on. The majority of errors arise due to retrieval of incorrect contextual logic rules or missing required logic rules by the background knowledge retriever. Another typical failure mode is also the incorrect separation of claims and implications. Relatively infrequent however are inaccuracies in generation of logical representation from text. This indicates that the main bottleneck lies in correctly capturing the semantic structure of the natural language in logic representation rather than the translation of natural language into logic representation when provided with sufficient information about entities and properties in a sentence.

Representation-related challenges due to use of neural stages appear also in the GraphMERT framework (Belova et al. 2025), where errors occur during knowledge graph assembly. In particular, the helper LLM responsible for selecting and assembling tail entities may introduce inaccuracies. This creates failure modes such as tail incompleteness, where an incomplete token sequence is falsely accepted as a valid entity by the

helper LLM leading to truncated tail nodes, and tail vagueness, where the correct entity tokens are absent from the model’s top-k predictions and the helper LLM produces a contextually plausible but semantically weak completion that results in overly general or imprecise triples. Furthermore, neural models tend to prioritize frequent entities during prediction, which can cause rare but semantically important entities to be overlooked. Despite these issues, the ablation study shows that even when 75% of the seed knowledge graph is removed, the GraphMERT framework still performs around 3.86% better than a baseline LLM-generated knowledge graph, suggesting that the structured integration of the graph framework provides robustness despite imperfect extraction.

A similar pattern is observed in the translation of legal provisions into executable code, such as in the Catala-based approach for tax law (Lorenzo, Pietromatera, and Holzenberger 2025), where the central challenge is the faithful translation of legal language into code while preserving the intended legal meaning. In particular, legal rules often admit multiple possible interpretations, whereas programming languages require a single precise formulation. This shows that while overall the correctness of the symbolic reasoning is contingent on the correctness of the symbolic representation itself, the main bottlenecks differ across neuro-symbolic architectures.

Transparency of Reasoning Process

From the perspective of transparency, neuro-symbolic approaches offer significant advantages over purely neural models because the reasoning process can be explicitly traced. Systems based on formal reasoning frameworks such as SMT solvers, Prolog programs including various approaches of Chain of Code prompting from (Kant et al. 2025) or domain-specific languages like Catala allow intermediate reasoning steps to be inspected, thereby providing clear traceability from claim to conclusions. Similarly, approaches based on graph structures and the ADF-based reasoning framework (Collenette, Atkinson, and Bench-Capon 2023) or the Split-Up architecture (Zeleznikow and Stranieri 1995) introduce transparent reasoning structures through explicitly designed graph representations or graphical decision pipelines.

However one limitation of employing formal reasoning systems, also mentioned in (Cummins et al. 2025) is that such transparency may not necessarily be easily interpretable. While the reasoning steps are formally traceable, they are expressed in specialised symbolic languages, such as Prolog or Catala code, that many legal practitioners cannot easily read. Therefore InsurLE (Cummins et al. 2025) attempts to adress this issue by introducing a meta-language bridging the gap, although being a pure symbolic system for automated insurance contract understanding. Even when explanations are provided, as in the Split-Up system where pre-written warrants justify intermediate claims (Zeleznikow and Stranieri 1995), the explanatory value may be limited, since the system does not necessarily highlight the precise distinctions that lead to specific numerical outcomes. This raises the questions of whether highly fine-grained numerical predictions are necessary in legal reasoning by AI, or whether more principle-based explanations may be more suitable for legal AI argument construction.

Accounting for Interpretive Openness

A further important criterion for legal AI systems is their ability to account for interpretive openness. Legal reasoning often allows multiple plausible interpretation of the same set of facts, whereas computational formalisms tend to enforce a single representation which creates problems for approaches that translate legal reasoning into executable code or classical logical representations. Programming languages require precise definitions and therefore struggle to accommodate ambiguity or several interpretations. Similarly, systems that translate legal arguments into formal logic typically rely on a single logical representations of the underlying reasoning. The Split-Up framework explicitly acknowledges this challenge through the HIWLTCO (How I would like to come out) principle (Zeleznikow and Stranieri 1995), which highlights that two lawyers may construct different argumentations from the same factual situation in order to support opposing claims. Rather than attempting to try to model both perspectives in one system Split-Up suggests modeling them separately, for example by constructing distinct reasoning systems that represent the perspectives of different parties such as husband and wife in the domain of percentage split determination. This approach resembles multi-agent reasoning frameworks in which different agents represent competing interpretations of the same problem.

Interestingly one can find an attempt to address interpretive openness in the purely symbolic framework ON-LINE of (Valente, Breuker, and Brouwer 1999). It combines a formal ontology which is responsible for structured reasoning with explicit reference links to textual legal sources allowing to point to multiple interpretations and tracing back mentions to sources. Such a hybrid representation may therefore offer a natural way of incorporating interpretive openness into legal reasoning.

Easily update the Knowledge Base

Another important requirement for legal AI systems is the ability to update the underlying knowledge base when laws change. In fine-tuning based approaches like (Tan et al. 2024) and (Calanzone, Teso, and Vergari 2024) updates are particularly difficult because legal knowledge is embedded within the model parameters. When laws change, the model may continue to reproduce outdated legal information unless the system is retrained. It can therefore be challenging to ensure that old laws are completely removed from the model’s weights.

Graph based approaches offer greater flexibility since knowledge graphs allow inspection and modification of entities and relations, so outdated triples could be removed and replaced with updated legal relations without having to completely retrain the whole system. Symbolic reasoning such as (Sadowski and Chudziak 2025) and (Lalwani et al. 2025) and executable code programs like (Lorenzo, Pietromatera, and Holzenberger 2025) are similarly easy to update since legal rules are translated into formal logic or code and can be modified directly when the law changes. This also counts for (Wei et al. 2025), because although it is a training or fine-tuning approach the encoding of logic rules is done manually from legal texts and used in the logic module which steers the neural component to follow logic based-reasoning, so if a law changes, only the manual encoding of the law needs to be changed in First-Order logic.

However systems such as Split-Up (Zelezniakow and Stranieri 1995) or ADF-based frameworks (Collenette, Atkinson, and Bench-Capon 2023) face a different challenge because they were specifically manually designed by experts to the application domain and legal changes may require manual redesign or extension and in large areas of law maintaining such a reasoning structure could become complex and resource-intensive.

Consistency of Analysis and Outcome

In legal contexts, identical facts and applicable rules should lead to identical conclusions, ensuring predictability in decision-making. Symbolic reasoning frameworks generally perform well here, so systems based on formal logic, such as SMT solvers, Prolog programs or domain-specific languages like *Catala* execute reasoning procedures deterministically. Given the same facts and rule set, they will consistently produce the same reasoning path and outcome. Neuro symbolic systems are hybrids of a symbolic and neural component and inherit consistency of the symbolic component. Early stages in the pipeline used for entity extraction or rule generation, such as in (Belova et al. 2025), (Sadowski and Chudziak 2025), (Lalwani et al. 2025) rely on neural models to perform these tasks whose outputs may vary. As discussed in the above section of Methodical Correctness these are sources of failure modes since outputs of neural components and correctness vary.

4.2 Trade-Offs in Neural-Symbolic Systems and their Limitations

Trade-off between generalisability and transparency

Highly engineered approaches such as the ADF-based reasoning framework (Collenette, Atkinson, and Bench-Capon 2023) or the Split-Up system (Zelezniakow and Stranieri 1995) achieve quite a strong performance within their narrowly defined legal domains, determining percentage split under Australian family law or predicting admissibility of application to the European Court of Human Rights (ECtHR). This is mainly due to the fact that the reasoning structure is specifically tailored by experts to fit this application. This however limits generalisation to other areas of law without substantial, manual redesign. Systems like SOLAR (Sadowski and Chudziak 2025), Code Languages as in (Lorenzo, Pietromatera, and Holzenberger 2025), GraphMERT (Belova et al. 2025) or (Wei et al. 2025) also operate only in their designed domain, American tax law, French tax law, diabetes knowledge or Chinese lending cases, although they can answer a larger variety of questions. Large flexibility such as answering any kind of legal questions regarding the law in a specific country mostly

requires a neural model which in turn comes at the cost of reduced transparency over the reasoning process and methodological control. Neural models provide greater coverage of area, but offer weaker guarantees regarding correctness and interpretability. Consequently there is a tradeoff between the benefits of domain-specific transparency and methodological correctness and generalisation capabilities.

Limitation due to Data dependence

A limitation of many approaches in neuro-symbolic AI for law applications and neural law applications in general is their dependence on sufficient amounts of high-quality data. Systems that translate legal texts into executable representations, such as code require annotated training data containing natural language and representation in code (Lorenzo, Pietromatera, and Holzenberger 2025). Producing such datasets is particularly challenging in the legal domain, since annotation typically requires legal expertise and appropriate interpretation of legal statutes but also knowledge of the coding language to translate into. As a result, dataset creation can be very time-intensive and costly, often requiring legal experts. Additionally in the legal domain often large digital training corpora are not easily available or not in sufficient large enough amounts, especially outside of English-language legal systems.

Graph-based approaches such as the GraphMERT framework for example require an already existing high-quality seed knowledge graph containing necessary and descriptive relations in the domain to in turn produce a high-quality populated knowledge graph (Belova et al. 2025). Especially since the designed system does not add new relations itself and also if it did one would need human evaluation once again for verification on the quality or suitability of relations for the application domain.

A further limitation concerns the learning of reasoning from data itself. As noted in (Collenette, Atkinson, and Bench-Capon 2023) many machine learning models or neural models are trained on the written reasoning of court decisions after the outcome has already been determined. Consequently these sections serve to justify rather than represent reasoning process leading to the decision. Models trained on this data may therefore learn patterns to reconstruct justifications rather than genuinely inferring them from case facts

5 Future Work

There emerge several possible future research directions from the limit identifications in current neuro-symbolic legal reasoning systems.

The first point would be the development of evaluation metrics beyond considering only the whole end result since current metrics seen only assess whether the system arrives at a correct final answer. The quality of the intermediate representations used, especially concerning generated ontologies should also be included, potentially focusing on completeness, that all concepts can be represented and faithfulness, that all concepts are translated correctly when constructing from natural language to align with legal definitions. Development of specific metric frameworks for evaluation of generated code and knowledge graphs have been discussed in (Lorenzo, Pietromatera, and Holzenberger 2025) and (Belova et al. 2025).

A second major challenge is the expressive translation of legal text and case facts to formal representations. Future approaches could develop symbolic structures to better account for ambiguous language and open interpretations in law, thus overcoming the mapping of text into rigid logical or programmatic structures risking the loss of relevant legal meaning or oversimplification.

Another possible direction could be the revisiting of old approaches such as the Split-up system of (Zelevnikow and Stranieri 1995) and integrating modern neural network architectures to allow for a larger application domain and more differentiated warrant creation. Similarly ADF-based frameworks like (Collenette, Atkinson, and Bench-Capon 2023) could be extended to a full neural-symbolic system by modeling some of the decision heads in the graph as neural networks.

Finally additional research could improve the interpretability of symbolic structures used in neural-symbolic architectures. While formal representations such as logical rules or Prolog programs offer strong transparency they are hard for legal practitioners to understand, so developing a bridge interface between natural language and symbolic reasoning structure could improve practical usability of such neural symbolic systems in law.

6 Conclusions

Neural-symbolic AI offers a promising direction for automated legal reasoning by combining the strengths of neural language models and symbolic reasoning systems. Neural components provide natural language understanding and information extraction of legal texts, while symbolic components aid in structured reasoning, transparency, and consistency of outcomes. This combination is particularly relevant in the legal domain, where reasoning must be transparent and justifiable with legal rules.

This seminar paper surveyed a range of neural-symbolic approaches and organized them into three categories: systems that map legal information onto a specified formal structure, approaches that incorporate symbolic reasoning during model training or fine-tuning and architectures that combine neural and symbolic components by decomposing the task into smaller subtasks which are solved independently with a neural or symbolic component.

Across these approaches symbolic reasoning mechanisms typically introduce structured reasoning processes which can be verified, whereas the main sources of error arise during the neural stages responsible for translating natural language into formal representations resulting in strong dependence of the accuracy of neuro-symbolic legal reasoning systems on the quality of this representation step, as well as the quality and expressivity of the representation itself.

The seminar paper also highlighted an important tradeoff. Highly engineered symbolic structures can provide strong guarantees of transparency and methodological correctness but often lack flexibility and generalizability across different legal domains. In contrast, neural models offer coverage over a wide range of areas and adaptability but provide weaker guarantees regarding consistency of analysis and outcome and interpretability. Neural-symbolic architectures attempt to balance these competing qualities, yet many current systems are limited to narrowly defined legal tasks or require substantial effort to adapt to a different domain.

Another important challenge is the reliability on data. Training AI systems relies on sufficient amount of data which must be also of sufficient quality which is especially challenging in the legal domain since annotated data requires legal expertise which makes assembly of large datasets time-intensive and costly.

Overall, neural-symbolic AI represents a promising research direction for legal AI. Future work will most likely include improved engineered structures for translating legal language, development of evaluation metrics for assessing quality of intermediate reasoning steps such as the ontological structure and system designs which help understanding of formal representations used in these systems, such as code or logic for easier understanding. By addressing these challenges, future neural-symbolic systems may move closer to supporting practical applications in the legal domain.

Literature

- [1] Margarita Belova et al. “GraphMERT: Efficient and Scalable Distillation of Reliable Knowledge Graphs from Unstructured Data”. In: *arXiv preprint arXiv:2510.09580* (2025).
- [2] Diego Calanzone, Stefano Teso, and Antonio Vergari. “Logically consistent language models via neuro-symbolic integration”. In: *arXiv preprint arXiv:2409.13724* (2024).
- [3] Ilias Chalkidis et al. “LEGAL-BERT: The muppets straight out of law school”. In: *Findings of the association for computational linguistics: EMNLP 2020*. 2020, pp. 2898–2904.
- [4] Joe Collenette, Katie Atkinson, and Trevor Bench-Capon. “Explainable AI tools for legal reasoning about cases: A study on the European Court of Human Rights”. In: *Artificial Intelligence* 317 (2023), p. 103861.
- [5] John Cummins et al. “Computable contracts for insurance: Establishing an insurance-specific controlled natural language-insurle”. In: *Accessed: Feb 11* (2025), p. 2025.
- [6] Neel Guha et al. “Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models”. In: *Advances in neural information processing systems* 36 (2023), pp. 44123–44279.
- [7] Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. “A dataset for statutory reasoning in tax law entailment and question answering”. In: *arXiv preprint arXiv:2005.05257* (2020).
- [8] Manuj Kant et al. “Towards robust legal reasoning: Harnessing logical llms in law”. In: *arXiv preprint arXiv:2502.17638* (2025).
- [9] Abhinav Lalwani et al. “Autoformalizing natural language to first-order logic: A case study in logical fallacy detection”. In: *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. 2025, pp. 132–147.
- [10] Gabriele Lorenzo, Aldo Pietromatera, and Nils Holzenberger. “Translating Tax Law to Code with LLMs: A Benchmark and Evaluation Framework”. In: *Proceedings of the Natural Legal Language Processing Workshop 2025*. 2025, pp. 31–47.
- [11] Albert Sadowski and Jaroslaw A Chudziak. “On Verifiable Legal Reasoning: A Multi-Agent Framework with Formalized Knowledge Representations”. In: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 2025, pp. 2535–2545.
- [12] Sergio Servantez et al. “Chain of logic: Rule-based reasoning with large language models”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024, pp. 2721–2733.
- [13] Zhongxiang Sun et al. “Logic rules as explanations for legal case retrieval”. In: *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 2024, pp. 10747–10759.
- [14] Harry Surden. “Artificial intelligence and law: An overview”. In: *Ga. St. UL Rev.* 35 (2018), p. 1305.
- [15] Xiaoyu Tan et al. “Thought-Like-Pro: Enhancing Reasoning of Large Language Models through Self-Driven Prolog-based Chain-of-Thought”. In: *arXiv preprint arXiv:2407.14562* (2024).
- [16] André Valente, Joost Breuker, and Bob Brouwer. “Legal modeling and automated reasoning with ON-LINE”. In: *International Journal of Human-Computer Studies* 51.6 (1999), pp. 1079–1125.
- [17] Bin Wei et al. “An LLMs-based neuro-symbolic legal judgment prediction framework for civil cases”. In: *Artificial Intelligence and Law* (2025), pp. 1–35.
- [18] John Zeleznikow and Andrew Stranieri. “The split-up system: integrating neural networks and rule-based reasoning in the legal domain”. In: *Proceedings of the 5th international conference on Artificial intelligence and law*. 1995, pp. 185–194.
- [19] Zhi Zhou et al. “Lawgpt: A chinese legal knowledge-enhanced large language model”. In: *arXiv preprint arXiv:2406.04614* (2024).